Guía de desarrollo — Wilson

Wilson David Hernández Sánchez · M3 Solicitudes · M5 Mensajería · Offero, versión 4 · 30 de octubre de 2026 · Wilson responsable de base de datos

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M3 Solicitudes (complejidad media, ≈28 h) y M5 Mensajería (media-alta: ≈24 h completo, ≈14 h mínimo).
Rol transversal	**Responsable de la base de datos** (D-10): esquema y convenciones, revisión de todo PR con SQL y aplicación en `offero-prod`. Transición: Sprint 0 y 1 junto con Diego; desde el Sprint 2 lideras tú, Diego de respaldo. Iker sigue revisando la seguridad (RLS).
Carga estimada	≈66 h planificadas frente a ≈76 h disponibles (dejas la documentación y tomas la base de datos: ≈1 h por semana). Tiempo real en M5 (+8 h) solo si sobra.
Tu ventaja	Tienes un poco más de experiencia que los demás: sabes algo de JavaScript y tu app de tareas en Flask es muy parecida a M3.
Tu prioridad de la primera semana	Git en equipo y **empezar PostgreSQL**: vas a ser el responsable de la base de datos.
Desbloqueas a	Diego (requests para M4) e Iker (moderar solicitudes y reglas de reseñas).
Dependes de	Daniel (M0 y categories, mar 10 nov) y Diego (M1 núcleo, mar 10 nov; quotes, mar 17 nov).
Fechas clave	Primer PR mar 3 nov · requests mié 11 nov · formulario de solicitud dom 22 nov · listados y detalle dom 29 nov · **M3 terminado dom 6 dic** · messages en dev dom 13 dic · enviar y ver mensajes vie 18 dic · **M5 terminado vie 15 ene** · entrega vie 29 ene.

De Flask a Next.js: lo que ya sabes, con otro nombre

En Flask	En Next.js (Offero)
`@app.route("/tareas")`	Carpeta `src/app/solicitudes/` con su `page.js`
`/tareas/`	Carpeta `[requestId]`
Plantillas Jinja	Componentes de React (JSX)
`request.form` en un POST	Server Action que recibe los datos del formulario
SQLite o listas en memoria	PostgreSQL en Supabase
`login_required` o revisar permisos en la vista	Protección en el servidor + políticas RLS en la base de datos
`flask run`	`npm run dev`

Pregunta pendiente: ¿tu app de tareas guardaba los datos en una base de datos (por ejemplo SQLite) o en memoria? Si usaste SQL, díselo a Diego: cambia tu punto de partida en la ruta de aprendizaje.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Preparar tu computador; pareja con Daniel en M0; primer PR (mar 3); **práctica de PostgreSQL en dev**; diseño de la BD con Diego (jue 5); JS moderno, React y App Router.
1 · 9 – 15 nov	Tabla requests + RLS en pareja con Iker (mié 11); **revisar con Diego el SQL de categories, businesses y reports**.
2 · 16 – 22 nov	Formulario de nueva solicitud (dom 22); **desde aquí lideras la base de datos**: revisas y aplicas en prod el SQL de quotes.
3 · 23 – 29 nov	Listados por rol y detalle; ajuste del SELECT con Diego (dom 29).
4 · 30 nov – 6 dic	Editar y cancelar; prueba cruzada: **M3 terminado (dom 6 dic)**.
5 · 7 – 13 dic	Semana de finales (media carga): tabla messages y función can_message en dev, con Diego (dom 13).
6 · 14 – 18 dic	Enviar y ver mensajes, en pareja con Daniel (vie 18).
Descanso	19 dic – 3 ene.
7 · 4 – 15 ene	Bandeja, leídos, botones desde cotizaciones y contador de no leídos: **M5 terminado (vie 15 ene)**.
8 · 18 – 29 ene	Tiempo real solo si el PO lo aprueba (hasta vie 22); documentación final; presentación.

Ruta de aprendizaje (justo a tiempo)

Aprende cada tema justo antes de la tarea que lo necesita, no todo al principio. El orden importa: cada punto se apoya en el anterior.

Semana 1 (Sprint 0)

1. Git en equipo: clonar, rama, commits pequeños, push, pull request y actualizar tu rama con main (Anexo C).
2. **PostgreSQL desde cero:** qué es una tabla, tipos (uuid, text, integer, numeric, date, timestamptz, boolean), clave primaria y foránea.
3. **PostgreSQL:** restricciones NOT NULL, UNIQUE, CHECK y DEFAULT; SELECT con WHERE, ORDER BY y JOIN. Practica en el SQL Editor de offero-dev.
4. JavaScript moderno (ya sabes un poco): async/await, desestructuración, map y filter.
5. React y App Router: componentes, props, `page.js` y `layout.js`.

Sprint 1 (con Diego)

1. Convención del equipo: un archivo por cambio en `supabase/sql/` (Anexo D) y por qué nunca se edita uno ya aplicado.
2. RLS: USING frente a WITH CHECK; políticas para requests en pareja con Iker.
3. Triggers y funciones: entiende el trigger de perfiles y las funciones `is_admin()` y `get_my_role()` que hace Diego.
4. Cómo revisar un PR con SQL: tipos, restricciones, claves foráneas, ON DELETE, políticas presentes.
5. Server Actions y validación en el servidor (para tu formulario de M3).

Sprints 2–4 (lideras la BD)

1. Transacciones y funciones con privilegios (para revisar la función de aceptar de M4 con Iker).
2. Índices básicos: cuándo conviene un índice (columnas por las que se filtra mucho).
3. Vistas: por qué no tienen RLS y cómo revisarlas (`public_profiles`, promedio de reseñas).

4. Aplicar SQL en offero-prod de forma segura: el mismo archivo probado en dev, antes de fusionar, y anotarlo en el PR.

Sprint 5 (M5)

1. Políticas RLS que consultan otras tablas (requests y quotes).
2. Agrupar datos para armar la lista de conversaciones.
3. (Opcional) Supabase Realtime.

Tu rol transversal: responsable de la base de datos

- **Esquema:** cuidas que todas las tablas sigan las convenciones (Anexo D): uuid, created_at, claves foráneas, CHECK en estados, is_hidden en contenido moderable.
- **Revisión:** todo PR que traiga un archivo de `supabase/sql/` lo revisas tú. Iker revisa además las políticas RLS con su checklist.
- **Producción:** aplicas en `offero-prod` el mismo archivo probado en dev, **antes** de que se fusione el PR, y lo anotas en el PR. Diego es tu respaldo.
- **Bóveda:** mantienes la nota `Esquema de la base de datos` y revisas que cada dueño llene la nota de su tabla en `07 Base de datos`.
- **Transición:** en el Sprint 0 y el Sprint 1 revisas junto con Diego; desde el Sprint 2 lideras tú.
- **Cuidado:** en dev solo cambios que agregan; borrar o renombrar algo que otro usa se acuerda antes en Discord.

Módulo 3 — Publicación y gestión de solicitudes

Objetivo	El cliente publica solicitudes con lo que necesita, la cantidad, la ubicación, la fecha límite y la categoría; las administra (editar y cancelar), y los proveedores consultan las solicitudes abiertas.
Alcance	Mínimo = completo. Todo M3 es obligatorio y está en la ruta crítica: M4 se construye sobre tus solicitudes.
Fuera de alcance	Fotos adjuntas (solo si sobra tiempo), solicitudes privadas a un solo proveedor y búsqueda por texto.
Depende de	Diego (M1 núcleo, mar 10 nov); Daniel (categories, mar 10 nov).
Desbloquea a	Diego (M4: las cotizaciones se hacen sobre una solicitud).

Tabla `requests`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
client_id	uuid → profiles.id	Obligatorio: el cliente que la publica.
category_id	uuid → categories.id	Obligatorio.
title	text	Obligatorio, 5 a 120 caracteres.
description	text	Obligatorio, hasta 2000 caracteres.
quantity	integer	Mayor que 0.
location	text	Obligatorio: ciudad y barrio.
deadline	date	Al crear, debe ser una fecha futura (se valida en el servidor).
status	text	open, awarded o cancelled; por defecto open. awarded solo lo pone la función de aceptar de Diego.
is_hidden	boolean	Falso por defecto; solo el admin lo cambia.
created_at	timestamptz	Automático.

Políticas RLS de `requests`

Operación	Quién	Condición
SELECT	Cliente, proveedor, admin	El cliente ve todas las suyas; los proveedores ven las open no ocultas y, cuando exista quotes, también las que cotizaron; el admin ve todas.
INSERT	Cliente	Rol client; `client_id` es el usuario actual; status open.
UPDATE	Cliente dueño	Solo si está open: editar campos o cancelar. No puede cambiar `client_id` ni `is_hidden`, ni poner awarded.
DELETE	Nadie	Se cancela en vez de borrar: conserva el historial y no rompe las cotizaciones [SUPUESTO].

Regla de edición [SUPUESTO, decide el PO]: una solicitud solo se edita mientras no tenga cotizaciones, para que nadie cambie las condiciones después de que los proveedores cotizaron.

RLS controla filas, no columnas. Para que el dueño no cambie `client_id`, `is_hidden` ni el estado a awarded, el servidor no envía esos campos y la base lo bloquea con el mecanismo que Diego usa para el rol en profiles. Escribes las políticas en pareja con Iker; Diego revisa (todavía estás en la transición).

Rutas y archivos

- `src/app/solicitudes/nueva/page.js` — formulario (solo clientes).
- `src/app/solicitudes/page.js` — cliente: «Mis solicitudes» con su estado; proveedor: «Solicitudes abiertas» con filtro por categoría (reutiliza CategoryFilter de Daniel).
- `src/app/solicitudes/[requestId]/page.js` — detalle con acciones según el rol: el dueño edita, cancela y va a comparar cotizaciones (página de Diego); el proveedor ve el formulario de cotización (componente de Diego).
- `src/app/solicitudes/[requestId]/editar/page.js` [SUPUESTO: ruta nueva] — edición.
- `src/components/requests/` — RequestForm, RequestCard y RequestStatusBadge.
- `src/services/requests.js` — todas las consultas de solicitudes.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Primer PR de práctica: agrega tu fila a la tabla de integrantes del documento de contexto en `docs/`.	mar 3 nov	Tu PR fue revisado y fusionado sin conflictos.
2	Tabla requests + RLS en dev, en pareja con Iker.	mié 11 nov	Un cliente crea; un proveedor no puede crear; el cliente B no ve las solicitudes canceladas de A.
3	Formulario de nueva solicitud con Server Action y validaciones en el servidor: fecha futura, cantidad mayor que 0 y categoría existente.	dom 22 nov	Los datos inválidos muestran el error junto al campo y no se guardan.
4	Listado según el rol + filtro por categoría.	mar 24 nov	Cliente y proveedor ven listas distintas.
5	Detalle; 404 si no existe o no tiene permiso.	jue 26 nov	Un id ajeno o inventado muestra la página 404.
6	Con Diego: colocar QuoteForm en el detalle y ajustar el SELECT para los proveedores que cotizaron.	sáb 28 nov	Un proveedor sigue viendo una solicitud adjudicada que cotizó.
7	Editar (si está open y sin cotizaciones) y cancelar.	jue 3 dic	Una solicitud cancelada desaparece para los proveedores.
8	Prueba cruzada (la hace Daniel) y publicación con solicitudes reales del piloto.	dom 6 dic	Checklist de la Definition of Done completo (Anexo G).

Criterios de aceptación

- **Dado** un cliente con sesión, **cuando** publica una solicitud con datos válidos, **entonces** aparece en «Mis solicitudes» como abierta y los proveedores la ven en «Solicitudes abiertas».
- **Dado** un cliente, **cuando** pone una fecha límite pasada o una cantidad 0, **entonces** el servidor rechaza la solicitud y muestra el error junto al campo.
- **Dado** un proveedor, **cuando** intenta crear una solicitud desde la app o directo contra Supabase, **entonces** la operación falla.
- **Dado** el cliente A, **cuando** intenta editar o cancelar la solicitud del cliente B cambiando el id en la URL, **entonces** la operación falla.
- **Dado** un cliente, **cuando** cancela una solicitud abierta, **entonces** deja de aparecer para los proveedores y no recibe más cotizaciones.

Pruebas de permisos

Prueba como visitante, cliente A, cliente B, proveedor y admin: crear, ver la lista, ver el detalle, editar y cancelar. Anota el resultado esperado y el obtenido.

Errores típicos y cómo evitarlos

- Poner «use client» en toda la página: solo lo necesita el componente interactivo.
- Fechas con hora y zona horaria: guarda solo la fecha.
- La lista no se actualiza después de crear: hay que revalidar la ruta.
- Proteger ocultando botones en vez de verificar en el servidor y en RLS.
- Pegar código de la IA escrito para Flask, para el router viejo de Next.js (carpeta `pages/`) o para TypeScript.

Coordinación con el equipo

- Daniel: necesitas categories el mar 10 nov y su componente CategoryFilter.
- Diego: acuerden dónde va QuoteForm y el ajuste del SELECT para proveedores que cotizaron.
- Iker: coloca su ReportButton en el detalle de la solicitud y, en el Sprint 6 (dic), su ReviewForm cuando la solicitud esté adjudicada.

Módulo 5 — Mensajería y comunicación entre usuarios

Objetivo	Mensajes de texto entre el cliente de una solicitud y cada proveedor que la cotizó, guardados en PostgreSQL.
Alcance	**Mínimo + contador (hasta el vie 15 ene):** enviar, ver la conversación, bandeja, marcar como leídos, botones desde las cotizaciones y contador de no leídos. **Opcional (hasta el vie 22 ene, si el PO lo aprueba):** Supabase Realtime para ver mensajes sin recargar.
Fuera de alcance	Archivos o imágenes, chat sin solicitud, chats de grupo y avisos por correo.
Depende de	M3 (tuyo); quotes de Diego (mar 17 nov); vista public_profiles de Diego para mostrar nombres.
Desbloquea a	Nadie: es un módulo final.

Quién puede hablar [SUPUESTO, confirmar con el PO]: solo el cliente dueño de la solicitud y el dueño de un negocio que le envió una cotización. Evita mensajes no deseados y simplifica los permisos.

Tabla `messages`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
sender_id	uuid → profiles.id	Quien envía; debe ser el usuario actual.
receiver_id	uuid → profiles.id	Quien recibe; distinto de sender_id (CHECK).
request_id	uuid → requests.id	La solicitud de la conversación.
content	text	Obligatorio, 1 a 2000 caracteres.
created_at	timestamptz	Automático.
read_at	timestamptz	Vacío = no leído.

Políticas RLS de `messages`

Operación	Quién	Condición
SELECT	Participantes	Solo si eres `sender_id` o `receiver_id`. El admin no lee mensajes privados [SUPUESTO].
INSERT	Participantes válidos	`sender_id` es el usuario actual y la pareja cumple la regla (cliente de la solicitud ↔ dueño de un negocio que cotizó); la solicitud no está oculta.
UPDATE	Destinatario	Solo marcar `read_at`; nadie edita el contenido.
DELETE	Nadie	—

Para la regla de INSERT, crea una función de ayuda en la base (por ejemplo `can_message`) que reciba la solicitud y la otra persona. Revísala con Diego antes de fusionar.

Rutas y archivos

- `src/app/mensajes/page.js` — lista de conversaciones (solicitud + otra persona, último mensaje, no leídos); al elegir una se muestra la conversación y el formulario de envío.
- Botón «Enviar mensaje» en la comparación de cotizaciones (cliente → proveedor) y en «Mis cotizaciones» (proveedor → cliente): coordinar con Diego.
- `src/components/messages/` — ConversationList, MessageThread y MessageForm (carpeta nueva respecto al documento).

- `src/services/messages.js` — consultas y envío.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla messages + RLS + función can_message en dev (en pareja con Diego).	dom 13 dic	Un tercero no puede leer ni escribir en una conversación ajena.
2	Enviar y ver una conversación (en pareja con Daniel).	vie 18 dic	El mensaje aparece al recargar, con el nombre de la persona (sin correo).
3	Bandeja de conversaciones y marcar como leídos.	vie 8 ene	Al abrir una conversación, sus mensajes quedan leídos.
4	Botones de entrada desde las cotizaciones (con Diego).	mar 12 ene	Desde una cotización se abre la conversación correcta.
5	Contador de no leídos en el encabezado (coordina con Daniel).	mié 13 ene	El número baja al abrir la conversación.
6	Prueba cruzada (la hace Iker) y publicación con conversaciones reales. **M5 terminado.**	vie 15 ene	Checklist de la Definition of Done completo (Anexo G).
7	(Opcional, con aprobación del PO) Realtime.	vie 22 ene	Un mensaje nuevo aparece sin recargar; RLS sigue bloqueando a terceros.

Criterios de aceptación

- **Dado** un cliente con una solicitud cotizada por el proveedor A, **cuando** le escribe a A, **entonces** A ve el mensaje en /mensajes con el nombre del cliente y la solicitud.
- **Dado** un proveedor C que no cotizó esa solicitud, **cuando** intenta escribirle al cliente, **entonces** la operación falla.
- **Dado** un usuario que no participa en una conversación, **cuando** intenta leer sus mensajes, **entonces** no obtiene nada.
- **Dado** el destinatario, **cuando** abre la conversación, **entonces** los mensajes quedan leídos; el remitente no puede editar los mensajes enviados.

Errores típicos y cómo evitarlos

- Traer todos los mensajes y filtrarlos en el navegador: es una fuga de datos.
- Mostrar correos: usa la vista public_profiles.
- Pedir el nombre de cada mensaje con una consulta distinta: trae los nombres en una sola consulta.
- Activar Realtime sin políticas RLS correctas.

Coordinación con el equipo

- Diego: quotes debe existir en dev el mar 17 nov; acuerden los botones de entrada desde las cotizaciones.
- Iker: hace la prueba cruzada de M5 intentando leer y escribir en conversaciones ajenas.

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Primer despliegue dom 1 nov; M0 terminado sáb 7 nov; entornos y tablero listos; profiles y funciones de ayuda en dev
1	lun 9 – dom 15 nov	**M1 núcleo mar 10 nov**; registro e inicio de sesión; tablas categories, businesses, requests y reports con RLS
2	lun 16 – dom 22 nov	M1 terminado (mié 18); quotes en dev; «Mi negocio»; formulario de solicitud; categorías y usuarios en el panel
3	lun 23 – dom 29 nov	Primera cotización enviada; listados y detalle de negocios y solicitudes; reportes en el panel
4	lun 30 nov – dom 6 dic	**Núcleo completo (dom 6 dic):** solicitud → cotización → elegir con empresas reales; M2, M3, M4 y M7 mínimo terminados
5	lun 7 – dom 13 dic	Semana de finales: media carga [SUPUESTO]. Estabilizar el núcleo; messages y reviews en dev
6	lun 14 – vie 18 dic	Enviar mensajes y formulario de reseña; uso real del núcleo; avance con el profesor
—	sáb 19 dic – dom 3 ene	**Descanso: nadie trabaja**
7	lun 4 – vie 15 ene (2 semanas)	**MVP v1 (vie 15 ene):** los 8 módulos publicados en versión mínima
8	lun 18 – vie 29 ene (2 semanas)	Versiones completas hasta el **congelamiento vie 22 ene**; estabilización y presentación. **Entrega: vie 29 ene** [SUPUESTO]

Festivos en Colombia en días de trabajo: lun 2 nov, lun 16 nov, mar 8 dic y lun 11 ene. Sin subir horas: cada uno mantiene sus 12 h semanales (Diego 14 h), también en enero [SUPUESTO].

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo

#	Paso	Comando o acción	Cómo verificar
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si trae SQL, revisa Wilson (base de datos) e Iker (RLS); si toca autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR, lo revisa Wilson y Wilson (o Diego, de respaldo) lo aplica en prod antes de fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.

- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.
- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Wilson	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (bóveda de Obsidian)	Cada uno, su módulo; coordina Daniel	Actualiza tu nota de módulo y las de tus tablas al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;
- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.